Lecture No.8

SEARCHING AND SORTING

SEARCHING

- **The process of locating target data is known as *searching***
- **Searching is the process of finding the location of the target among a list of object**
- **The two basic search techniques are the following:**
  - **Sequential search**
  - **Binary search**

Search **and insert algorithm**

- **One of the most popular applications of search is while adding a record in the collection of records**
- **While adding, the record is searched by key and if not present, it is inserted in the collection**
- **Such a technique of searching the record and inserting it if not found is known as *search and insert algorithm***

Search **and insert algorithm**

- **One of the most popular applications of search is while adding a record in the collection of records**
- **While adding, the record is searched by key and if not present, it is inserted in the collection**
- **Such a technique of searching the record and inserting it if not found is known as *search and insert algorithm***

SEARCH TECHNIQUES

- **Sequential search**
- **Binary search**
- **Fibonacci search**
- **Hashed search**
- **Index sequential search**

Sequential Search

- **A sequential search begins with the first available record and proceeds to the next available record repeatedly until we find the target key or conclude that it is not found**
- **Sequential search is also called as *linear search***
- **Sequential search is used when the list is not sorted**

Sequential Search

- **Figure shows a sample sequential unordered data and traces the search for the target data of 89**

Index	0	1	2	3	4	5	6	7	8
Element	23	12	9	10	11	89	78	66	88

Analysis of Sequential Search

- **Average complexity is the sum of comparisons for each position of the target data divided by n. Hence,**
- **average number of comparisons**
- **= (1 + 2 + 3 + … + n)/n**
- **= (Σn)/n**
- **= ((n(n + 1))/2) × 1/n**
- **= (n + 1)/2**
- **The worst-case complexity = n**
- **The best-case complexity = 1**

Advantages and Disadvantages of Sequential Search

- **Advantages:**
  - **A simple and easy method**
  - **Efficient for small lists**
  - **Suitable for unsorted data**
  - **Suitable for storage structure, which does not support direct access to data, for example, magnetic tape**
  - **Best case is one comparison, worst case is n comparisons, and average case is (n+ 1)/2 comparisons**
  - **Complexity is in the order of n denoted as O(n)**

- **Disadvantages:**
  - **Highly efficient for large data**
  - **Other search techniques such as binary search are found more suitable than sequential search for ordered data**

Variations of Sequential Search

- **There are three such variations:**
  - **Sentinel search**
  - **Probability search**
  - **Ordered list search**

Sentinel search

- **The algorithm ends either when the target is found or when the last element is compared**
- **The algorithm can be modified to eliminate the end of list test by placing the target at the end of list as just one additional entry**
- **This additional entry at the end of the list is called as *sentinel***

Probability search

- **In probability search, the elements that are more probable are placed at the beginning of the array and those that are less probable are placed at the end of the array**

Ordered list search

- **When elements are ordered, binary search is preferred**
- **However, when data is ordered and is of smaller size, sequential search with small change is preferred than binary search**

Binary Search

- **In binary search, the target is first searched at the mid of the list**
- **As the list is sorted (ascending or descending), if the target is not found at the mid, then it is searched either in upper half or in lower half**
- **If the list is in ascending order and if the target is smaller than the element at mid, then it is searched in upper half, else the target is searched in the lower half using binary search**

Tail recursive

- **A recursive function is said to be tail recursive if there are no pending operations to be performed on return from a recursive call**
- **Tail recursion is also used to return the value of the last recursive call as the value of the function**
- **Tail recursion is advantageous as the amount of information which must be stored during computation is independent of the number of recursive calls**

Advantages and Disadvantages of Binary Search

- **Advantages:**
  - **Suitable for sorted data**
  - **Efficient for large lists**
  - **Suitable for storage structure that supports direct access to data**
  - **Time complexity is O(log2(*n*))**

Disadvantages

- **Not usable for unsorted data**
- **Not usable for storage structure that do not support direct access to data, for example, magnetic tape and linked list**
- **Inefficient for small lists**

Time Complexity Analysis

T(*n*) = *O(log2n)* The time complexity can be written as a recurrence relation as:

$$T(n) = \begin{cases} T(1) & n = 1 \\ T(n/2) + c & n > 1 \end{cases}$$

Fibonacci Search

- **Fibonacci search changes the binary search algorithm slightly**
- **Instead of halving the index for a search, a Fibonacci number is subtracted from it**
- **The Fibonacci number to be subtracted decreases as the size of the list decreases**
- **Note that Fibonacci search sorts a list in a non decreasing order**
- **Fibonacci search starts searching the target by comparing it with the element at *Fk*th location**

The different cases for the search are as follows:

Case 1: if equal the search terminates;

Case 2: if the target is greater and F_1 is 1, then the search terminates with an unsuccessful search;

else the search continues at the right of list with new values of low, high, and mid as

$\texttt{mid} = \texttt{mid} + F_2$, $F_1 = F_{k-4}$ and $F_2 = F_{k-5}$

Case 3: if the target is smaller and F_2 is 0, then the search terminates with unsuccessful search;

else the search continues at the left of list with new values of low, high, and mid as

$\texttt{mid} = \texttt{mid} - F_2$, $F_1 = F_{k-3}$ and $F_2 = F_{k-4}$

The search continues by either searching at the left of mid or at the right of mid in the list.

Time Complexity of Fibonacci Search

- **Fibonacci search is more efficient than binary search for large- sized lists**
- **However, it is inefficient in case of small lists**
- **The number of comparisons is of the order of *n, and the time complexity is O(log(n))***

Indexed Sequential Search

- **An index file can be used to effectively overcome the problem associated with sequential files and to speed up the key search**
- **The simplest indexing structure is the single-level one: a file whose records are pairs key and a pointer), where the pointer is the position in the data file of the record with the given key**
- **Only a subset of data records, evenly spaced along the data file, is indexed to mark the intervals of data records**

Indexed Sequential Search:

Record position	Emp. no.	Name	Occupation
1	100	Saurabh	Developer
2	500	Abolee	Project head
3	300	Shweta	Developer
4	200	Vaishali	Project head
5	400	Santosh	Developer

Index file of Table

Emp. no. (key)	Record position
100	1
200	4
300	3
400	5
500	2

Indexed Sequential Search

- **Searching a record from this index file involves the following issues:**
  - **Index file is ordered, so the searching in index file can be done using the binary search method**
  - **Search is successful if we found the target element in the index**
  - **Record position is used to access the details of that record from data file**

SORTING

- **Sorting is the operation of arranging the records of a table according to the key value of each record, or it can be defined as the process of converting an unordered set of elements to an ordered set of elements**
- **Sorting is a process of organizing data in a certain order to help retrieve it more efficiently**

Internal and External Sorting

- **Any sort algorithm that uses main memory exclusively during the sorting is called as *internal sort algorithm***
- **Internal sorting is faster than external sorting**

Internal Sorting

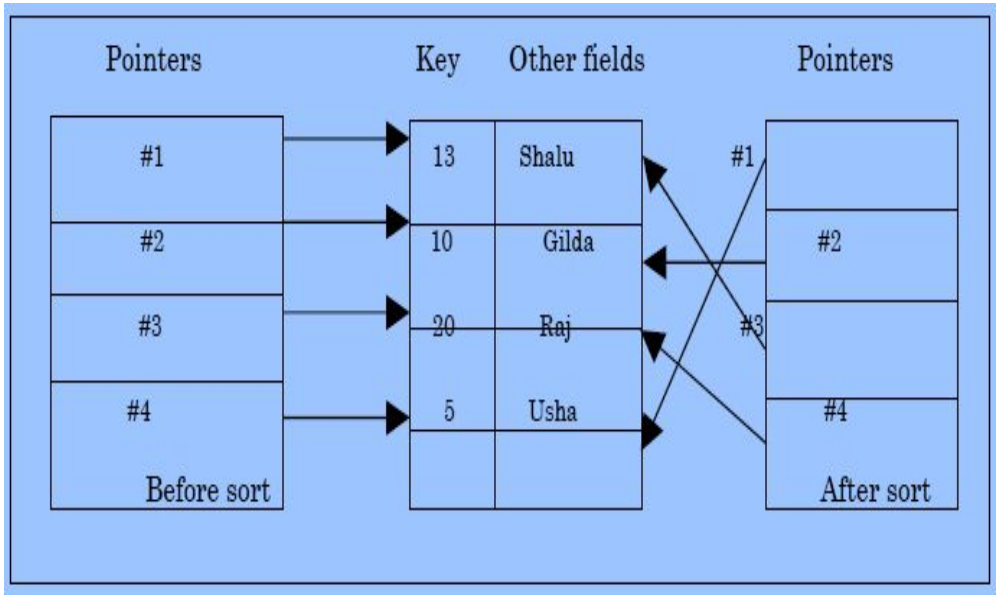
- **The various internal sorting techniques are the following:**
  - **Bubble sort**
  - **Selection sort**
  - **Insertion sort**
  - **Quick sort**
  - **Shell sort**
  - **Heap sort**
  - **Radix sort**
  - **Bucket sort**

External Sorting

- Any **sort algorithm** that uses external memory, such as tape or disk, during the sorting is called as **external sort algorithm**

- **Merge sort** **is used in external sorting**

Sorting with pointers



General Sort Concepts

- **Sort Order :**
  - **Data can be ordered either in ascending order or in descending order**
  - **The order in which the data is organized, either ascending order or descending order, is called *sort order***

Sort Stability

- **A sorting method** is said to be stable if at the end of the method, identical elements occur in the same relative order as in the original unsorted set

Name	Uma	Saurabh	Sanika	Kasturi	Ashish	Harsha	Lelo
Marks	80	90	93	95	83	90	83

The stable sort method will sort the sequence as

Name	Kasturi	Sanika	Saurabh	Harsha	Ashish	Lelo	Uma
Marks	95	93	90	90	83	83	80

Whereas the unstable sort method may sort the same sequence a

Name	Kasturi	Sanika	Harsha	Saurabh	Lelo	Ashish	Uma
Marks	95	93	90	90	83	83	80

Sort Efficiency

- **Sort efficiency is a measure of the relative efficiency of a sort**
- **It is usually an estimate of the number of comparisons and data movement required to sort the data**

Passes

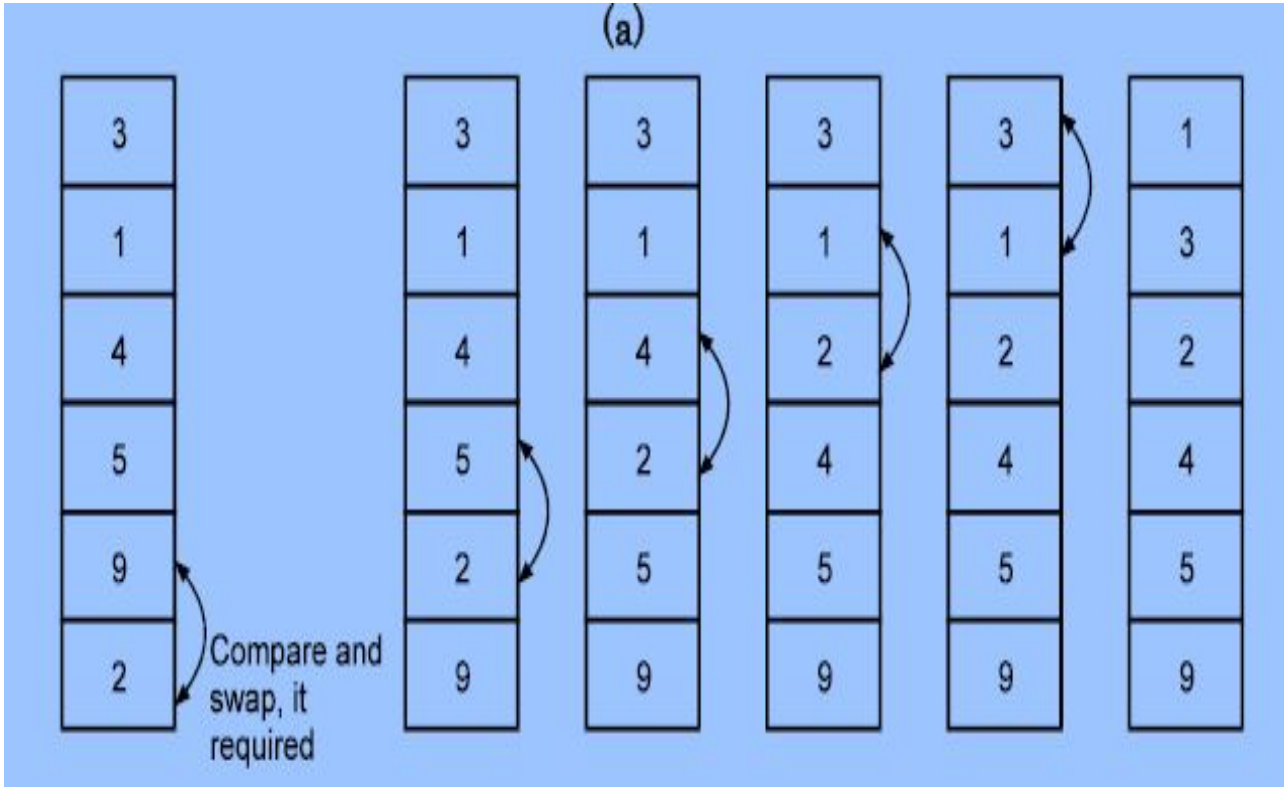
- **During the sorted process, the data is traversed many times**
- **Each traversal of the data is referred to as a *sort pass***
- **In addition, the characteristic of a sort pass is the placement of one or more elements in a sorted list**

Bubble Sort

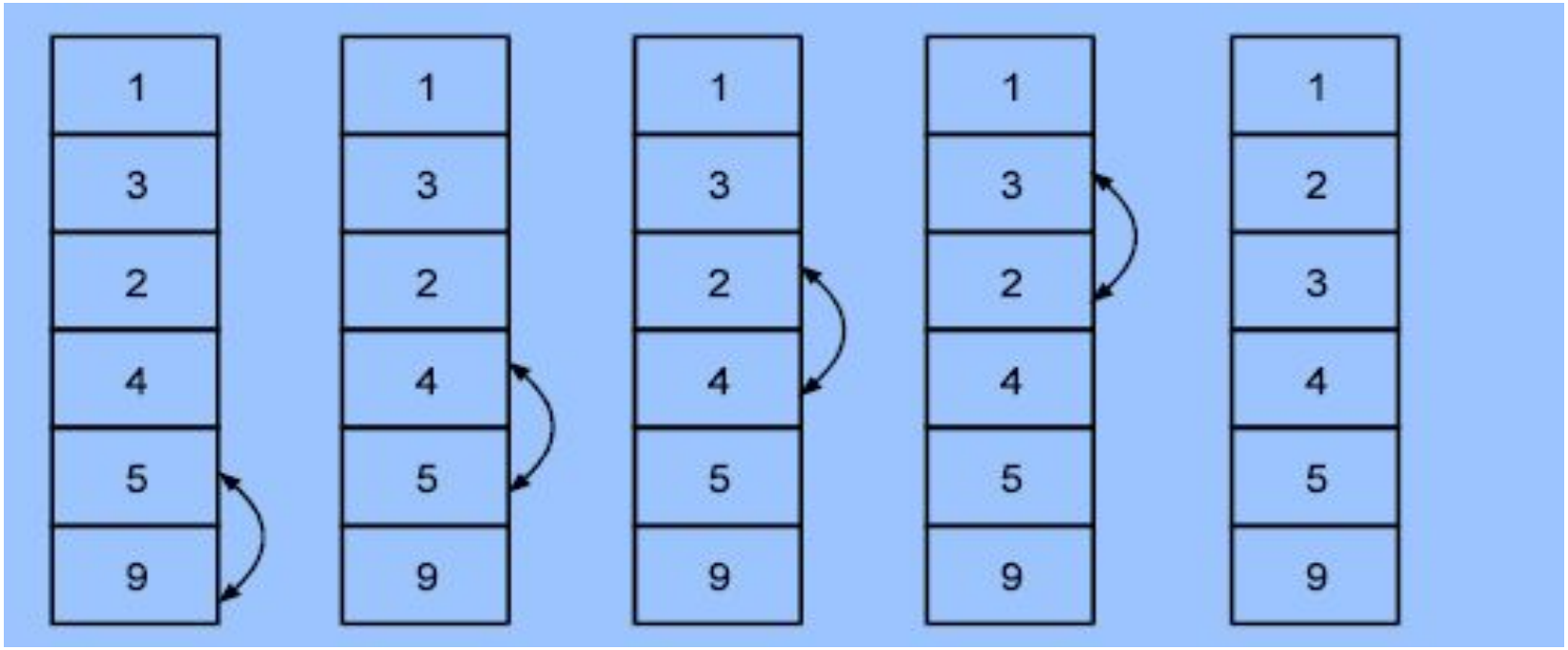
- **The bubble sort works by comparing each item in the list with the item next to it and swapping them if required**
- **The algorithm repeats this process until it makes a pass all the way through the list without swapping any items (in other words, all items are in the correct order)**
- **This causes larger values to 'bubble' to the end of the list while smaller values 'sink' towards the beginning of the list**
- **In brief, the bubble sort derives its name from the fact that the smallest data item bubbles up to the top of the sorted array**

Best, Worst and Average Cases

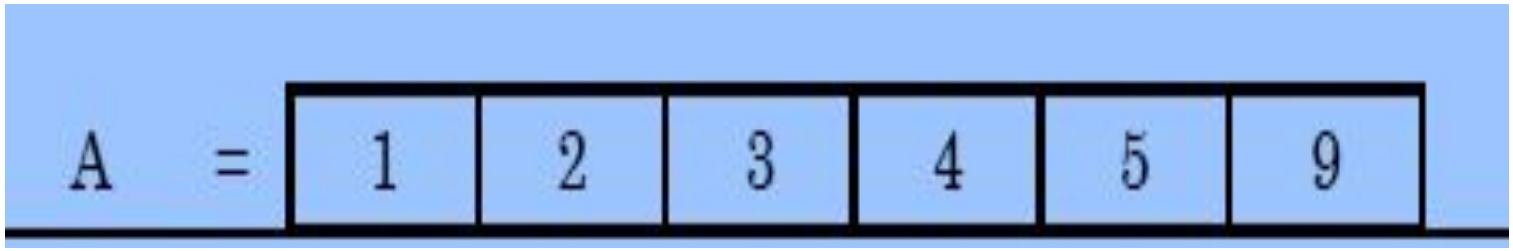
- **Worst Case :**
  - **Complexity of the algorithm is the function defined by the maximum number of steps taken on any instance of size n**
- **Best Case:**
  - **Complexity of the algorithm is the function defined by the minimum number of steps taken on any instance of size n**
- **Average Case:**
  - **Complexity of the algorithm is the function defined by an average number of steps taken on any instance of size n**



Demonstrating bubble sort (a) Pass 1 (i = 1)



(b) Pass 2 (i = 2)



c: After pass (n – 1) (i = 5),
the resultant sorted array

Analysis of Bubble Sort

- **The time complexity for each of the cases is given by the following:**
  - **Average-case complexity = O(*n2)***
  - **Best-case complexity = O(*n2)***
  - **Worst-case complexity = O(*n2)***

Insertion Sort

- **The insertion sort works just like its name suggests—it inserts each item into its proper place in the final list**
- **The simplest implementation of this requires two list structures: the source list and the list into which the sorted items are inserted**

Analysis of Insertion Sort

- **If the data is initially sorted, only one comparison is made on each pass so that the sort time complexity is O(n)**
- **The number of interchanges needed in both the methods is on the average (n2)/4, and in the worst cases is about (n2)/2**

Selection *Sort*

- **The selection sort algorithms construct the sorted sequence, one element at a time, by adding elements to the sorted sequence in order**

- **At each step, the next element to be added to the sorted sequence is selected from the remaining elements**

- In this method, we sort a set of unsorted elements in two steps
- In the first step, find the smallest element in the structure
- In the second step, swap the smallest element with the element at the first position
- Then, find the next smallest element and swap with the element at the second position
- Repeat these steps until all elements get arranged at proper positions

	Original unsorted array						
Elements	76	67	36	55	23	14	6
Index	0	1	2	3	4	5	6

Index	0	1	2	3	4	5	6	I	minpos
	76	67	36	55	23	14	**6**	0	6
Pass 1	6	**67**	36	55	23	**14**	76	1	5
Pass 2	6	14	**36**	55	**23**	67	76	2	4
Pass 3	6	14	23	**55**	**36**	67	76	3	4
Pass 4	6	14	23	36	**55**	67	76	4	4
Pass 5	6	14	23	36	55	**67**	76	5	5
Sorted array	**6**	**14**	**23**	**36**	**55**	**67**	**76**		

Analysis of Selection Sort

Average case: $O(n^2)$ Best case: $O(n^2)$ Worst case: $O(n^2)$

Quick sort

- **Quick sort is based on divide-and-conquer strategy**
- **Quick sort is thus in-place, divide-and-conquer based massively recursive sort technique**
- **This technique reduces unnecessary swaps and moves the element at great distance in one move**

Quick sort

- **The recursive algorithm consists of four steps:**
  - **If there is one or less element in the array to be sorted, return immediately**
  - **Pick an element in the array to serve as a 'pivot' usually the left-most element in the list)**
  - **Partition the array into two parts—one with elements smaller than the pivot and the other with elements larger than the pivot by traversing from both the ends and performing swaps if needed**
  - **Recursively repeat the algorithm for both partitions**

Choice of Pivot

- **We can choose any entry in the list as the pivot**
- **The choice of the first entry as pivot is popular but often a poor choice**

Analysis of Quick Sort

- **The average complexity =O(n logn)**
- **The worst-case time complexity = O(n2)**

Merge Sort

- **The most common algorithm used in external sorting is the merge sort**
- **Merging is the process of combining two or more sorted files into the third sorted file**
- **We can use a technique of merging two sorted lists**
- **Divide and conquer is a general algorithm design paradigm that is used for merge sort**

Merge Sort

- Time Complexity T(n) = $O(n \log n)$

Heap Sort

- **Heap sort is one of the fastest sorting algorithms, which achieves the speed as that of quick sort and merge sort**
- **The advantages of heap sort are as follows: it does not use recursion, and it is efficient for any data order**
- **It achieves the worst-case bounds better than those of quick sort**
- **And for the list, it is better than merge sort since it needs only a small and constant amount of space apart from the list being sorted**

Heap Sort

- **The steps for building heap sort are as follows:**
  - **Build the heap tree**
  - **Start delete heap operation storing each deleted element at the end of the heap array**

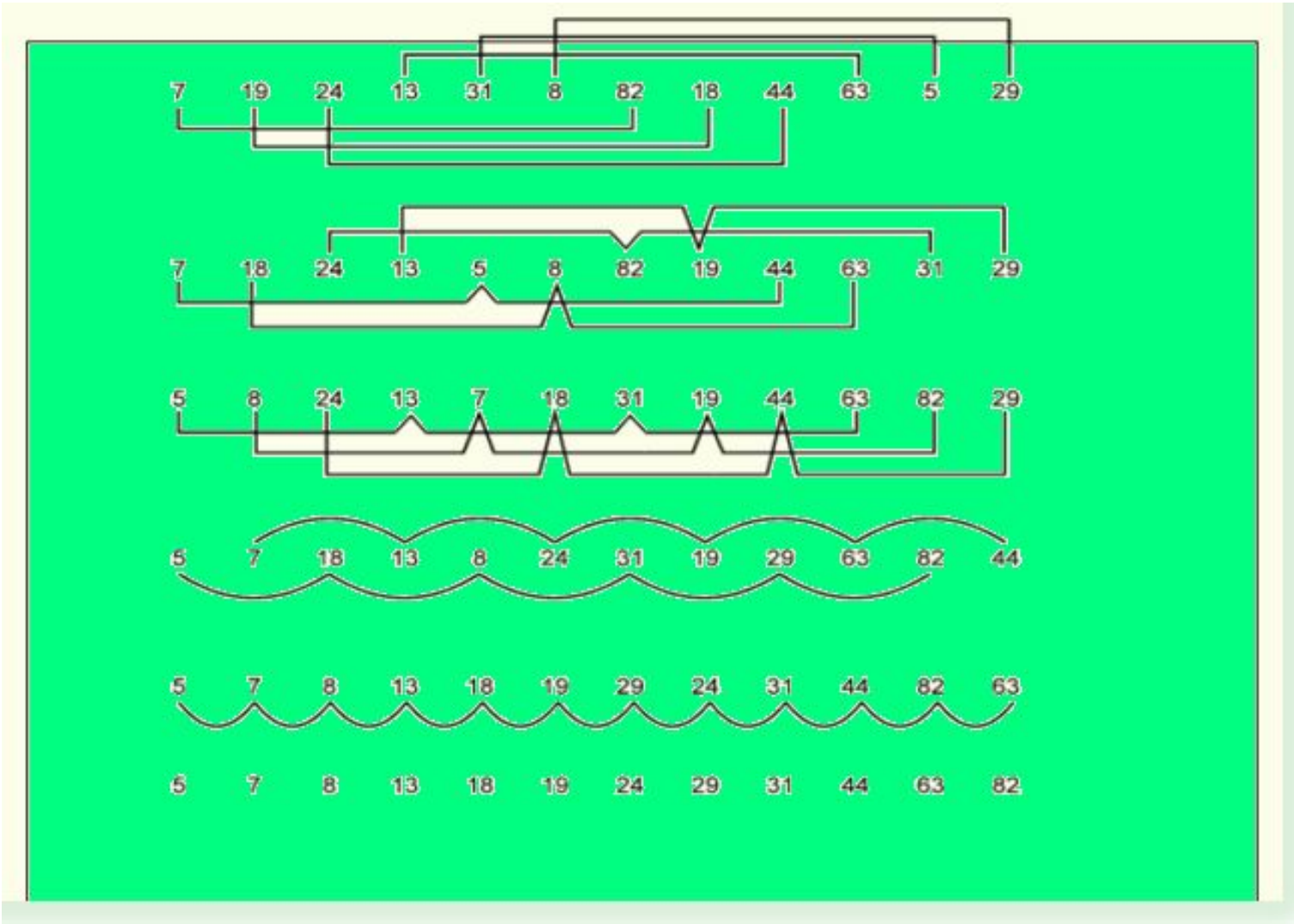
Heap Sort

- **ALGORITHM**
- **1. Build a heap tree with a given set of data**
  - **(a) Delete root node from heap**
  - **(b) Rebuild the heap after deletion**
  - **(c) Place the deleted node in the output**
- **Continue with step (2) until the heap tree is empty**

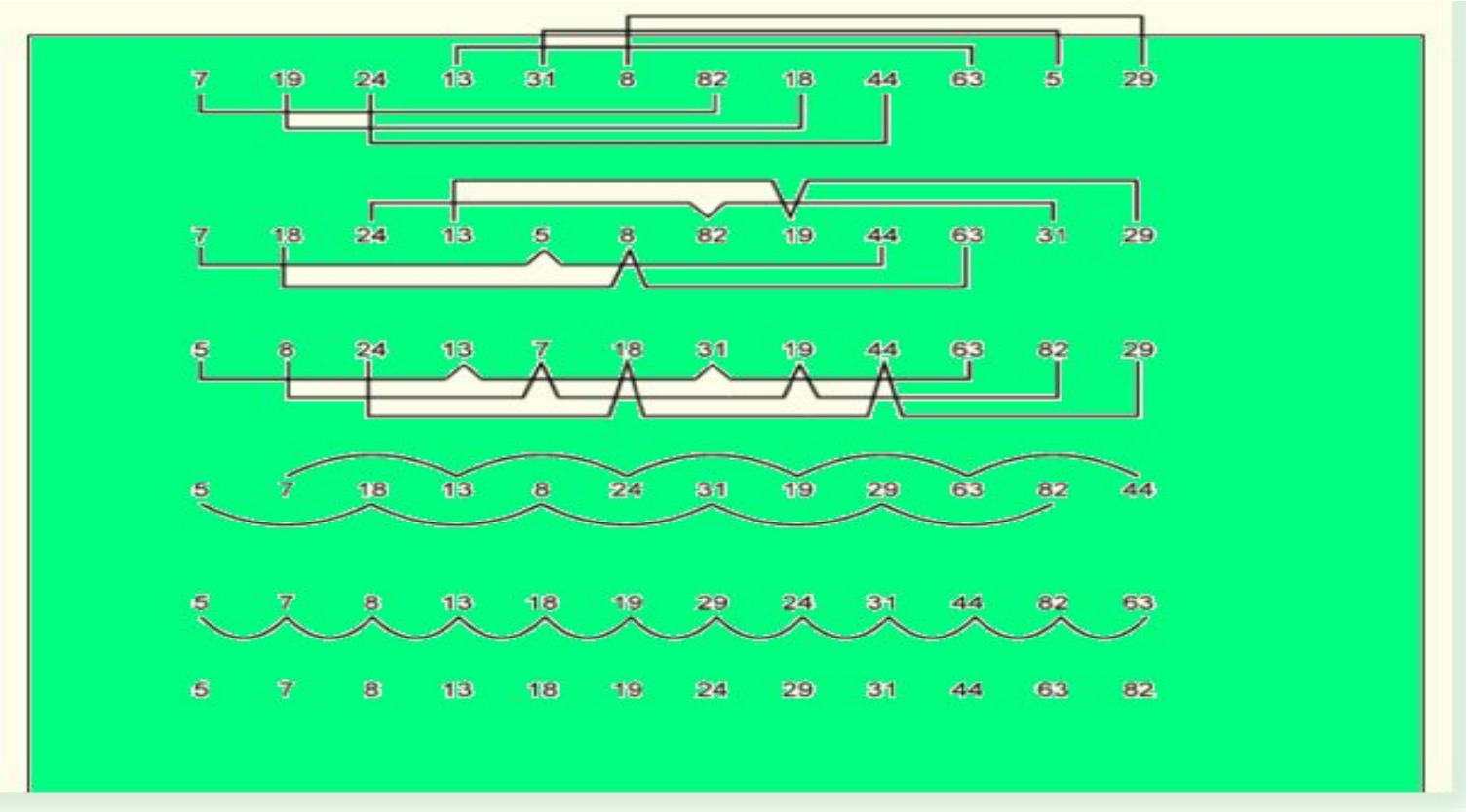
Analysis of Heap **Sort**

- **The time complexity is stated as follows:**
  - **Best case O(n logn)**
  - **Average case O(n logn)**
  - **Worst case O(n logn)**

Shell Sort


7 19 24 13 31 8 82 18 44 63 5 29
7 18 24 13 5 8 82 19 44 63 31 29
5 8 24 13 7 18 31 19 44 63 82 29
5 7 18 13 8 24 31 19 29 63 82 44
5 7 8 13 18 19 29 24 31 44 82 63
5 7 8 13 18 19 24 29 31 44 63 82

Shell Sort


7 19 24 13 31 8 82 18 44 63 5 29
7 18 24 13 5 8 82 19 44 63 31 29
5 8 24 13 7 18 31 19 44 63 82 29
5 7 18 13 8 24 31 19 29 63 82 44
5 7 8 13 18 19 29 24 31 44 82 63
5 7 8 13 18 19 24 29 31 44 63 82

Bucket Sort

- **Bucket sort is possibly the simplest distribution sorting algorithm**
- **In bucket sort, initially, a fixed number of buckets are selected**

Bucket Sort

- **For example, suppose that we are sorting elements from the set of integers in the interval [0, *m − 1]. The bucket sort uses m buckets or counters***

- ***The ith* counter/bucket keeps track of the number of occurrences of the *ith element of the list***

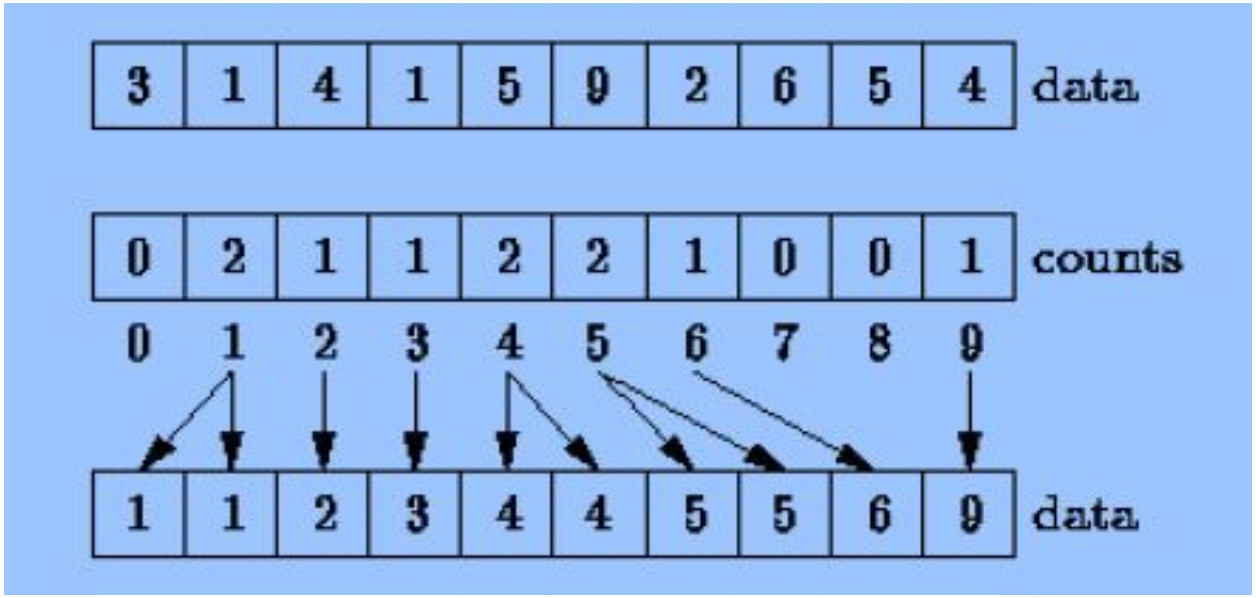


Illustration of how this is done for $m = 9$

Radix Sort

- **Radix sort is a generalization of bucket sorting**
- **Radix sort works in three steps:**

  - **Distribute all elements into *m buckets***
  - ***Here m is a suitable integer, for example, to sort* decimal numbers with radix 10**
  - **We take 10 buckets numbered as 0, 1, 2, …, 9**
  - **For sorting strings, we may need 26 buckets, and so on**
  - **Sort each bucket individually**
  - **Finally, combine all buckets**

External Sort or File Sort

- **There are numerous algorithms used to perform sorts external to the computer's main memory**
- **Among the many external sort methods, the poly phase sort is more efficient in terms of speed and utilization of resources**
- **However, it is more complicated, and therefore, Merge sort technique is commonly used for external sort and is suitable for internal sort too**

COMPARISON OF ALL SORTING METHODS

Sorting method	Technique in brief	Best case	Worst case	Memory requirement	Is stable	Pros	Cons
Bubble sort	Repeatedly stepping through the list to be sorted, comparing each pair of adjacent items and swapping them if they are in the wrong order	$O(n^2)$	$O(n^2)$	No extra space needed	Yes	1. A simple and easy method 2. Efficient for small lists $n > 100$	Highly inefficient for large data

Selection sort	Finds the minimum value in the list and then swaps it with the value in the first position, repeats these steps for the remainder of the list (starting at the second position and advancing each time)	$O(n^2)$	$O(n^2)$	No extra space needed	Yes	1. Recommended for small files 2. Good for partially sorted data	Inefficient for large lists

Insertion sort	Every repetition of insertion sort removes an element from the input data, inserts it into the correct position in the already sorted list until no input elements remain. The choice of which element to remove from the input is arbitrary	$O(n)$	$O(n^2)$	No extra space needed	Yes	1. Relatively simple and easy to implement 2. Good for almost sorted data	Inefficient for large lists

Quick sort	Picks an element, called a pivot, from the list Reorders the list so that all elements with values less than the pivot come before the pivot, whereas all elements with values greater than the pivot come after it (equal values can go either way). After this partitioning, the pivot is in its final position. This is called the partition operation Recursively sorts the sub-list of the lesser elements and the sub-list of the greater elements	$O(n\log_2 n)$	$O(n^2)$	No extra space needed	Yes	1. Extremely fast 2. Inherently recursive	Very complex algorithm

Shell Sort	It is a generalization of insertion sort, which exploits the fact that insertion sort works efficiently on input that is already almost sorted. It improves on insertion sort by allowing the comparison and	$O(n^{1.5})$	$O(n\log^2 n)$	No extra space needed	No	1. It is faster than a quick sort for small arrays 2. Its speed and simplicity makes it a good choice in practice	Slower for sufficiently big arrays

Radix sort (most significant digit)	Numbers are placed at proper location by processing individual digits and by comparing individual digits that share the same significant position	O(n)	O(n)	Extra space proportional to n is needed	Yes	1. Radix sort is very simple, and fast 2. In-Place, Recursive and one of the fastest sorting algorithms for numbers or strings of letters	Radix sort can also take more space than other sorting algorithms since in addition to the array that will be sorted, there needs to be a sub-list for each of the possible digits or letters

Merge sort	Conceptually, a merge sort works as follows: If the list is of length 0 or 1, then it is already sorted. Otherwise, the algorithm divides the unsorted list into two sub-lists of about half the size Then, it sorts each sub-list recursively by reapplying the merge sort and then merges the two sub-lists back into one sorted list	$O(n\log_2 n)$	$O(n\log_2 n)$	Extra space proportional to n is needed	Yes	1. Good for external file sorting 2.Can be applied to files of any size	1. It requires twice the memory of the heap sort because of the second array used to store the sorted list. 2. It is recursive, which can make it a bad choice for applications that run on machines with limited memory

Heap sort	Heap sort begins by building a heap out of the data set, and then removing the largest item and placing it at the end of the partially sorted array. After removing the largest item, it reconstructs the heap, removes the largest remaining item, and places it in the next open position from the end of the partially sorted array. This is repeated until there are no items left in the heap and the sorted array is full	$O(n\log_2 n)$	$O(n\log_2 n)$	No extra space needed	No	1. Advantageous as it does not use recursion and that heap sort works just as fast for any data order. That is, there is basically no worst-case scenario 2. Heaps work well for small tables and the tables where changes are infrequent	Do not work well for most large tables

Summary

- Searching means locating a target element in the list. There are basically two search techniques: sequential (also known as linear search) and binary search.
- Sequential search is used when the list is not sorted, and binary search is preferred when the list is in sorted order.
- The variations of linear search include the following: sentinel search and probabilistic search.
- In sentinel search, the check for the end of list is avoided by placing the target at the end of list.
- The probability search orders the list by placing the most probable elements at the beginning of the list.

Summary

- In binary search, the target is first searched at the mid of the list. As the list is sorted (ascending or descending), if the target is not found at the mid, then it is searched either in upper half or in lower half
- If the list is in ascending order and if the target is smaller than the element at mid, then it is searched in upper half, else the target is searched in the lower half using binary search
- The time complexity of linear is O(*n), whereas it is O(log2n) for binary search.*

Summary

- If the equal targets maintain their relative input order in the output, then the sorting method is called as the stable sorting method
- Internal sort techniques are broadly classified as insertion, selection, and exchange
- Insertion sorting include insertion sort and shell sort. Selection sorting methods are selection and heap sort
- Heap sort is an improved version of selection sort. Bubble sort and quick sort are two exchange sort techniques
- Quick sort is faster and handles arrays of heterogeneous data fairly efficiently
- The shell short is more efficient than the bubble sort, selection sort, and insertion sort
- Sorting of larger files that cannot fit in main memory is best accomplished by external sorting techniques such as the merge sort